

Proyecto 2 – Análisis y Diseño de Algoritmos II

¿Dónde pongo mi concierto?

Andrés Eduardo Narváez Cañas 2259545 - Jose David Mármol Otero 2266370
Universidad del Valle – Sede Tuluá

Resumen—Encontrar dónde poner un concierto en un cuadrado de $N \times N$ km con M ciudades, usando distancia Manhattan, de forma equitativa para todas. Se modeló como un problema de optimización con restricciones. La app en Python con Tkinter genera código MiniZinc a partir de la entrada; el código se copia y ejecuta en el IDE de MiniZinc con Gecode. La función objetivo minimiza primero la distancia máxima y luego la dispersión entre ciudades.

Index Terms—Optimización, MiniZinc, Distancia Manhattan, Minimax, Programación con Restricciones.

I. MODELO DEL PROBLEMA

I-A. Descripción del problema

El Valle del Cauca se modela como un plano discreto de $N \times N$ km con esquina inferior izquierda en $(0, 0)$. Hay M ciudades en posiciones enteras (cx_i, cy_i) con $0 \leq cx_i, cy_i \leq N$. El concierto debe quedar lo más parejo posible para todas, sin caer en ninguna ciudad y sin favorecer a una sobre las demás. “Favorecer” lo entendemos como que una ciudad quede notablemente más cerca que el resto.

I-B. Formulación formal

Variables de decisión:

$$x \in \{0, \dots, N\}, \quad y \in \{0, \dots, N\} \quad (1)$$

Distancia Manhattan a cada ciudad i :

$$d_i = |x - cx_i| + |y - cy_i|, \quad \forall i \in \{1, \dots, M\} \quad (2)$$

No coincidencia:

$$\forall i: \quad (x \neq cx_i) \vee (y \neq cy_i) \quad (3)$$

Variables auxiliares:

$$\maxDist = \max(d_1, \dots, d_M), \quad \minDist = \min(d_1, \dots, d_M) \quad (4)$$

Función objetivo:

$$\min f(x, y) = W \cdot \maxDist + (\maxDist - \minDist) \quad (5)$$

I-C. Justificación del peso W

$\maxDist$ va primero porque lo más importante es que ninguna ciudad quede demasiado lejos. El segundo término minimiza la dispersión. Para que el primer término siempre domine, se necesita $W > 2N$ (la dispersión nunca supera $2N$). El mínimo válido para cualquier N es $W = 2N + 1$. Usamos $W = 1000$, válido para $N \leq 499$.

II. IMPLEMENTACIÓN

Tres archivos de Python:

parser.py: Lee N , M y las ciudades. Para separar nombre de coordenadas, toma los **dos últimos tokens** como X e Y y el resto como nombre, lo que maneja correctamente nombres con espacios como “Río Frío”.

minizinc_generator.py: Construye el modelo MiniZinc completo como string: constantes, arreglos, variables, restricciones y función objetivo. Listo para copiar y pegar.

main.py: Tkinter con TextArea de entrada (con ejemplo precargado), botón “Generar Modelo MiniZinc” y TextArea de salida. Muestra mensaje de error si la entrada es inválida.

III. PRUEBAS

III-A. Prueba 1 – Caso base ($N = 12$, $M = 5$)

Cuadro I
CASO BASE

Ciudad	X	Y
Palmira	2	3
Cali	10	2
Buga	11	0
Tuluá	0	3
Río Frío	1	2

Resultado: $(5, 1)$. Distancias: Palmira 5, Cali 6, Buga 7, Tuluá 7, Río Frío 5. $\maxDist = 7$, $\minDist = 5$, dispersión=2.

III-B. Prueba 2 – Esquinas simétricas ($N = 10$, $M = 4$)

Cuadro II
CIUDADES EN LAS ESQUINAS

Ciudad	X	Y
A	0	0
B	10	0
C	0	10
D	10	10

Resultado: $(5, 5)$, $\maxDist = \minDist = 10$, dispersión=0. El centro queda a distancia Manhattan 10 de las cuatro esquinas ($|5-0| + |5-0| = 10$). Caso perfecto, no se favorece a nadie.

III-C. Prueba 3 – Ciudades en línea ($N = 20$, $M = 3$)

Cuadro III
CIUDADES COLINEALES

Ciudad	X	Y
P1	0	10
P2	10	10
P3	20	10

Resultado: (10,11), $maxDist = 11$, $minDist = 1$, dispersión=10. El punto ideal (10,10) está bloqueado por P2. Al moverse a (10,11) queda a distancia 1 de P2 y 11 de P1 y P3; la brecha no se puede cerrar sin subir $maxDist$.

III-D. Prueba 4 – Clúster denso ($N = 8$, $M = 6$)

Cuadro IV
CLÚSTER DENSO

Ciudad	X	Y
C1	3	3
C2	3	4
C3	4	3
C4	4	4
C5	3	5
C6	5	3

Resultado: (4,5), $maxDist = 3$, $minDist = 1$. Los puntos interiores están ocupados, el concierto va a la periferia. Aun así todas las ciudades quedan a distancia máxima 3.

III-E. Prueba 5 – Ciudad aislada ($N = 20$, $M = 4$)

Cuadro V
UNA CIUDAD AISLADA

Ciudad	X	Y
X1	1	1
X2	2	1
X3	1	2
Lejana	19	19

Resultado: (10,10), $maxDist = 18$, $minDist = 17$, dispersión=1.

IV. ANÁLISIS DE LAS PRUEBAS

IV-A. Resumen de resultados

Cuadro VI
RESUMEN COMPARATIVO

Prueba	N	M	$maxDist$	$minDist$	Disp.
Base	12	5	7	5	2
Esquinas	10	4	10	10	0
Línea	20	3	11	1	10
Clúster	8	6	3	1	2

IV-B. Tiempos de ejecución en MiniZinc

Tiempos aproximados medidos con el cronómetro interno de MiniZinc usando Gecode:

Cuadro VII
TIEMPOS VARIANDO N Y M

N	M	Tiempo (ms)
10	4	< 50
12	5	< 50
20	4	< 50
50	5	~ 80
100	10	~ 150
200	10	~ 400
200	20	~ 900
500	10	~ 1200

El tiempo crece principalmente con N (dominio más grande). Duplicar M con $N = 200$ duplica el tiempo aproximadamente. Para $N \leq 100$ y $M \leq 10$ resuelve en menos de 200 ms.

IV-C. Distribución, empates y casos raros

La distribución espacial de las ciudades es el factor que más afecta el resultado, por encima del tamaño del grid N .

Simetría perfecta: cuando las ciudades están distribuidas simétricamente respecto al centro, el punto central queda equidistante de todas. La dispersión resulta 0 porque $maxDist = minDist$. Es el caso ideal, ilustrado en la Prueba 2 con las cuatro esquinas.

Ciudad aislada: cuando existe un grupo de ciudades cercanas entre sí y una sola ciudad muy lejana, esa ciudad domina el problema. El criterio minimax obliga al concierto a alejarse del cluster y acercarse a la ciudad solitaria para reducir la distancia máxima, como se observó en la Prueba 5 donde *Lejana*(19,19) jaló el concierto hacia el centro (10,10).

Ciudades en línea con punto medio bloqueado: cuando las ciudades están alineadas y el punto geoméricamente ideal está ocupado por una ciudad, la restricción de no coincidencia obliga al concierto a desplazarse, lo que incrementa la dispersión. Esto ocurrió en la Prueba 3, donde (10,10) estaba ocupado por P2 y el concierto debió ubicarse en (10,11).

Función objetivo y empates: la función objetivo del modelo es:

$$\min f(x, y) = W \cdot maxDist + (maxDist - minDist) \quad (6)$$

El término $W \cdot maxDist$ garantiza que reducir la distancia máxima sea siempre la prioridad absoluta. El peso $W = 1000$ supera cualquier valor posible del spread, acotado por $2N$, por lo que $maxDist$ nunca puede ser compensado por un spread menor. El segundo término ($maxDist - minDist$) actúa únicamente como desempate: entre todos los puntos que logran el mismo $maxDist$ mínimo, se elige el de menor dispersión, es decir, el más equitativo entre los candidatos óptimos.

Sin embargo, pueden existir dos o más puntos con exactamente el mismo $maxDist$ y el mismo spread, produciendo el mismo valor de la función objetivo. En ese caso ambos puntos son igualmente óptimos bajo el criterio del modelo

y cualquiera constituye una ubicación válida y justa para el concierto. MiniZinc con Gecode devuelve el primero que encuentra durante la búsqueda Branch and Bound, lo cual depende del orden interno de exploración del solver y no es determinista. Otro solver o una ejecución con distinta configuración podría devolver el otro punto, pero ambos resultados son correctos.

Caso $M = 1$: con una sola ciudad el spread es siempre 0, ya que existe una única distancia y por tanto $\text{maxDist} = \text{minDist}$. El modelo elige el punto más cercano a esa ciudad sin coincidir con ella. El modelo ejecuta sin errores, pero el resultado carece de significado desde el enunciado, que define equidad como la ausencia de favoritismo entre múltiples ciudades.

V. CONCLUSIONES

El modelo escala bien para el alcance del problema: con $N \leq 200$ y hasta 20 ciudades MiniZinc resuelve en menos de un segundo. El cuello de botella es el tamaño del dominio (N^2 posiciones), no las restricciones.

La función objetivo funciona, pero $W = 1000$ solo es válido para $N \leq 499$; parametrizar $W = 2N + 1$ elimina esa limitación. Más relevante aún: la distribución espacial importa mucho más que N . Una ciudad aislada mueve el concierto al centro aunque el resto esté en una esquina, y la restricción de no coincidencia puede empeorar la dispersión significativamente cuando el punto óptimo cae en una ciudad.

Lo más valioso fue tener que generar el modelo MiniZinc en vez de resolver el problema directamente. Eso obligó a entender cómo expresar restricciones formalmente y cómo traducir “equidad” a algo que un solver pueda optimizar, que es una forma de pensar muy distinta a la programación común.